JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ClassLoader

java.lang.Object
    java.lang.ClassLoader

Direct Known Subclasses:

SecureClassLoader

```
public abstract class ClassLoader
extends Object
```

A class loader is an object that is responsible for loading classes. The class `ClassLoader` is an abstract class. Given the binary name of a class, a class loader should attempt to locate or generate data that constitutes a definition for the class. A typical strategy is to transform the name into a file name and then read a "class file" of that name from a file system.

Every `Class` object contains a `reference` to the `ClassLoader` that defined it.

`Class` objects for array classes are not created by class loaders, but are created automatically as required by the Java runtime. The class loader for an array class, as returned by `Class.getClassLoader()` is the same as the class loader for its element type; if the element type is a primitive type, then the array class has no class loader.

Applications implement subclasses of `ClassLoader` in order to extend the manner in which the Java virtual machine dynamically loads classes.

In addition to loading classes, a class loader is also responsible for locating resources. A resource is some data (a "`.class`" file, configuration data, or an image for example) that is identified with an abstract '/'-separated path name. Resources are typically packaged with an application or library so that they can be located by code in the application or library. In some cases, the resources are included so that they can be located by other libraries.

The `ClassLoader` class uses a delegation model to search for classes and resources. Each instance of `ClassLoader` has an associated parent class loader. When requested to find a class or resource, a `ClassLoader` instance will usually delegate the search for the class or resource to its parent class loader before attempting to find the class or resource itself.

Class loaders that support concurrent loading of classes are known as *parallel capable* class loaders and are required to register themselves at their class initialization time by invoking the `ClassLoader.registerAsParallelCapable` method. Note that the `ClassLoader` class is registered as parallel capable by default. However, its subclasses still need to register themselves if they are parallel capable. In environments in which the delegation model is not strictly hierarchical, class loaders need to be parallel capable, otherwise class loading

can lead to deadlocks because the loader lock is held for the duration of the class loading process (see `loadClass` methods).

Run-time Built-in Class Loaders

The Java run-time has the following built-in class loaders:

- Bootstrap class loader. It is the virtual machine's built-in class loader, typically represented as `null`, and does not have a parent.
- Platform class loader. The platform class loader is responsible for loading the *platform classes*. Platform classes include Java SE platform APIs, their implementation classes and JDK-specific run-time classes that are defined by the platform class loader or its ancestors. The platform class loader can be used as the parent of a `ClassLoader` instance.

  To allow for upgrading/overriding of modules defined to the platform class loader, and where upgraded modules read modules defined to class loaders other than the platform class loader and its ancestors, then the platform class loader may have to delegate to other class loaders, the application class loader for example. In other words, classes in named modules defined to class loaders other than the platform class loader and its ancestors may be visible to the platform class loader.
- System class loader. It is also known as *application class loader* and is distinct from the platform class loader. The system class loader is typically used to define classes on the application class path, module path, and JDK-specific tools. The platform class loader is the parent or an ancestor of the system class loader, so the system class loader can load platform classes by delegating to its parent.

Normally, the Java virtual machine loads classes from the local file system in a platform-dependent manner. However, some classes may not originate from a file; they may originate from other sources, such as the network, or they could be constructed by an application. The method `defineClass` converts an array of bytes into an instance of class `Class`. Instances of this newly defined class can be created using `Class.newInstance`.

The methods and constructors of objects created by a class loader may reference other classes. To determine the class(es) referred to, the Java virtual machine invokes the `loadClass` method of the class loader that originally created the class.

For example, an application could create a network class loader to download class files from a server. Sample code might look like:

```
ClassLoader loader = new NetworkClassLoader(host, port);
Object main = loader.loadClass("Main", true).newInstance();
     . . .
```

The network class loader subclass must define the methods `findClass` and `loadClassData` to load a class from the network. Once it has downloaded the bytes that make up the class, it should use the method `defineClass` to create a class instance. A sample implementation is:

```
class NetworkClassLoader extends ClassLoader {
    String host;
    int port;

    public Class findClass(String name) {
        byte[] b = loadClassData(name);
        return defineClass(name, b, 0, b.length);
    }

    private byte[] loadClassData(String name) {
        // load the class data from the connection
         . . .
    }
}
```

Binary names

Any class name provided as a `String` parameter to methods in `ClassLoader` must be a binary name as defined by *The Java Language Specification*.

Examples of valid class names include:

```
"java.lang.String"
"javax.swing.JSpinner$DefaultEditor"
"java.security.KeyStore$Builder$FileBuilder$1"
"java.net.URLClassLoader$3$1"
```

Any package name provided as a `String` parameter to methods in `ClassLoader` must be either the empty string (denoting an unnamed package) or a fully qualified name as defined by *The Java Language Specification*.

See *Java Language Specification*:

6.7 Fully Qualified Names and Canonical Names
13.1 The Form of a Binary

Since:

1.0

See Also:

`resolveClass(Class)`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`ClassLoader()`	Creates a new class loader using the `ClassLoader` returned by the method `getSystemClassLoader()` as the parent class loader.
protected	`ClassLoader (ClassLoader parent)`	Creates a new class loader using the specified parent class loader for delegation.
protected	`ClassLoader (String name, ClassLoader parent)`	Creates a new class loader of the specified name and using the specified parent class loader for delegation.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
void	`clearAssertionStatus()`	Sets the default assertion status for this class loader to `false` and discards any package defaults or class assertion status settings associated with the class loader.
		Deprecated.

protected final Class<?>	**defineClass**(byte[] b, int off, int len)	Replaced by defineClass(String, byte[], int, int)
protected final Class<?>	**defineClass** (String name, byte[] b, int off, int len)	Converts an array of bytes into an instance of class Class.
protected final Class<?>	**defineClass** (String name, byte[] b, int off, int len, ProtectionDomain protect	Converts an array of bytes into an instance of class Class, with a given ProtectionDomain.
protected final Class<?>	**defineClass** (String name, ByteBuffer b, ProtectionDomain protect	Converts a ByteBuffer into an instance of class Class, with the given ProtectionDomain.
protected Package	**definePackage** (String name, String specTitle, String specVersion, String specVendor, String implTitle, String implVersion, String implVendor, URL sealBase)	Defines a package by name in this ClassLoader.
protected Class<?>	**findClass**(String name)	Finds the class with the specified binary name.
protected Class<?>	**findClass** (String moduleName, String name)	Finds the class with the given binary name in a module defined to this class loader.
protected String	**findLibrary** (String libname)	Returns the absolute path name of a native library.
protected final Class<?>	**findLoadedClass** (String name)	Returns the class with the given binary name if this loader has been recorded by the Java

		virtual machine as an initiating loader of a class with that binary name.
protected URL	**findResource**(String name)	Finds the resource with the given name.
protected URL	**findResource**(String moduleName, String name)	Returns a URL to a resource in a module defined to this class loader.
protected Enumeration<URL>	**findResources**(String name)	Returns an enumeration of URL objects representing all the resources with the given name.
protected final Class<?>	**findSystemClass**(String name)	Finds a class with the specified binary name, loading it if necessary.
protected Object	**getClassLoadingLock**(String className)	Returns the lock object for class loading operations.
final Package	**getDefinedPackage**(String name)	Returns a Package of the given name that has been defined by this class loader.
final Package[]	**getDefinedPackages**()	Returns all of the Packages that have been defined by this class loader.
String	**getName**()	Returns the name of this class loader or null if this class loader is not named.

protected Package	**getPackage**(String name)	**Deprecated.** If multiple class loaders delegate to each other and define classes with the same package name, and one such loader relies on the lookup behavior of `getPackage` to return a `Package` from a parent loader, then the properties exposed by the `Package` may not be as expected in the rest of the program.
protected Package[]	**getPackages**()	Returns all of the `Packages` that have been defined by this class loader and its ancestors.
final ClassLoader	**getParent**()	Returns the parent class loader for delegation.
static ClassLoader	**getPlatformClassLoader**()	Returns the platform class loader.
URL	**getResource**(String name)	Finds the resource with the given name.
InputStream	**getResourceAsStream**(String name)	Returns an input stream for reading the specified resource.
Enumeration<URL>	**getResources**(String name)	Finds all the resources with the given name.
static ClassLoader	**getSystemClassLoader**()	Returns the system class loader.
static URL	**getSystemResource**(String name)	Find a resource of the specified name from the search path used to load classes.
static InputStream	**getSystemResourceAsStrea**(String name)	Open for reading, a resource of the specified

		name from the search path used to load classes.
static Enumeration<URL>	**getSystemResources**(String name)	Finds all resources of the specified name from the search path used to load classes.
final Module	**getUnnamedModule**()	Returns the unnamed Module for this class loader.
final boolean	**isRegisteredAsParallelCa**	Returns true if this class loader is registered as parallel capable, otherwise false.
Class<?>	**loadClass**(String name)	Loads the class with the specified binary name.
protected Class<?>	**loadClass**(String name, boolean resolve)	Loads the class with the specified binary name.
protected static boolean	**registerAsParallelCapabl**	Registers the caller as parallel capable.
protected final void	**resolveClass**(Class<?> c)	Links the specified class.
Stream<URL>	**resources**(String name)	Returns a stream whose elements are the URLs of all the resources with the given name.
void	**setClassAssertionStatus**(String className, boolean enabled)	Sets the desired assertion status for the named top-level class in this class loader and any nested classes contained therein.
void	**setDefaultAssertionStatu**	Sets the default assertion status for this class loader.
	(boolean enabled)	

void	**setPackageAssertionStatu**(String packageName, boolean enabled)	Sets the package default assertion status for the named package.
protected final void	**setSigners**(Class<?> c, Object[] signers)	Sets the signers of a class.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

ClassLoader

```
protected ClassLoader(String name,
                      ClassLoader parent)
```

Creates a new class loader of the specified name and using the specified parent class loader for delegation.

API Note:

If the parent is specified as `null` (for the bootstrap class loader) then there is no guarantee that all platform classes are visible.

Parameters:

`name` - class loader name; or `null` if not named

`parent` - the parent class loader

Throws:

`IllegalArgumentException` - if the given name is empty.

Since:

9

ClassLoader

```
protected ClassLoader(ClassLoader parent)
```

Creates a new class loader using the specified parent class loader for delegation.

API Note:

If the parent is specified as `null` (for the bootstrap class loader) then there is no guarantee that all platform classes are visible.

Parameters:

`parent` - The parent class loader

Since:

1.2

ClassLoader

```
protected ClassLoader()
```

Creates a new class loader using the `ClassLoader` returned by the method `getSystemClassLoader()` as the parent class loader.

Method Details

getName

```
public String getName()
```

Returns the name of this class loader or `null` if this class loader is not named.

API Note:

This method is non-final for compatibility. If this method is overridden, this method must return the same name as specified when this class loader was instantiated.

Returns:

name of this class loader; or `null` if this class loader is not named.

Since:

9

loadClass

```
public Class<?> loadClass(String name)
                   throws ClassNotFoundException
```

Loads the class with the specified binary name. This method searches for classes in the same manner as the `loadClass(String, boolean)` method. It is invoked by the Java virtual machine to resolve class references. Invoking this method is equivalent to invoking `loadClass(name, false)`.

Parameters:

`name` - The binary name of the class

Returns:

The resulting `Class` object

Throws:

`ClassNotFoundException` - If the class was not found

loadClass

```
protected Class<?> loadClass(String name,
                             boolean resolve)
                      throws ClassNotFoundException
```

Loads the class with the specified binary name. The default implementation of this method searches for classes in the following order:

1. Invoke `findLoadedClass(String)` to check if the class has already been loaded.
2. Invoke the `loadClass` method on the parent class loader. If the parent is `null` the class loader built into the virtual machine is used, instead.
3. Invoke the `findClass(String)` method to find the class.

If the class was found using the above steps, and the `resolve` flag is true, this method will then invoke the `resolveClass(Class)` method on the resulting `Class` object.

Subclasses of `ClassLoader` are encouraged to override `findClass(String)`, rather than this method.

Unless overridden, this method synchronizes on the result of `getClassLoadingLock` method during the entire class loading process.

Parameters:

`name` - The binary name of the class

`resolve` - If `true` then resolve the class

Returns:

The resulting `Class` object

Throws:

`ClassNotFoundException` - If the class could not be found

getClassLoadingLock

```
protected Object getClassLoadingLock(String className)
```

Returns the lock object for class loading operations.

API Note:

This method allows parallel capable class loaders to implement finer-grained locking schemes such that multiple threads may load classes concurrently without deadlocks. For non-parallel-capable class loaders, the `ClassLoader` object is synchronized on during the class loading operations.

Class loaders with non-hierarchical delegation should be registered as parallel capable to prevent deadlocks.

Implementation Requirements:

If this `ClassLoader` object is registered as parallel capable, this method returns a dedicated object associated with the specified class name. Otherwise, this method returns this `ClassLoader` object.

Parameters:

`className` - The name of the to-be-loaded class

Returns:

the lock for class loading operations

Throws:

`NullPointerException` - If registered as parallel capable and `className` is `null`

Since:

1.7

See Also:

`loadClass(String, boolean)`

findClass

```
protected Class<?> findClass(String name)
                      throws ClassNotFoundException
```

Finds the class with the specified binary name. This method should be overridden by class loader implementations that follow the delegation model for loading classes, and will be invoked by the `loadClass` method after checking the parent class loader for the requested class.

Implementation Requirements:

The default implementation throws `ClassNotFoundException`.

Parameters:

`name` - The binary name of the class

Returns:

The resulting `Class` object

Throws:

`ClassNotFoundException` - If the class could not be found

Since:

1.2

findClass

```
protected Class<?> findClass(String moduleName,
                             String name)
```

Finds the class with the given binary name in a module defined to this class loader. Class loader implementations that support loading from modules should override this method.

API Note:

This method returns `null` rather than throwing `ClassNotFoundException` if the class could not be found.

Implementation Requirements:

The default implementation attempts to find the class by invoking `findClass(String)` when the `moduleName` is `null`. It otherwise returns `null`.

Parameters:

`moduleName` - The module name; or `null` to find the class in the unnamed module for this class loader

`name` - The binary name of the class

Returns:

The resulting `Class` object, or `null` if the class could not be found.

Since:

9

defineClass

```
@Deprecated(since="1.1")
protected final Class<?> defineClass(byte[] b,
                                     int off,
                                     int len)
                              throws ClassFormatError
```

Deprecated.
Replaced by `defineClass(String, byte[], int, int)`

Converts an array of bytes into an instance of class `Class`. Before the `Class` can be used it must be resolved. This method is deprecated in favor of the version that takes a binary name as its first argument, and is more secure.

Parameters:

`b` - The bytes that make up the class data. The bytes in positions `off` through `off+len-1` should have the format of a valid class file as defined by *The Java Virtual Machine Specification*.

`off` - The start offset in `b` of the class data

`len` - The length of the class data

Returns:

The `Class` object that was created from the specified class data

Throws:

`ClassFormatError` - If the data did not contain a valid class

`IndexOutOfBoundsException` - If either `off` or `len` is negative, or if `off+len` is greater than `b.length`.

`SecurityException` - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class, or if an attempt is made to define a class in a package with a fully-qualified name that starts with "`java.`".

See Also:

`loadClass(String, boolean)`,
`resolveClass(Class)`

defineClass

```
protected final Class<?> defineClass(String name,
                                     byte[] b,
                                     int off,
                                     int len)
                              throws ClassFormatError
```

Converts an array of bytes into an instance of class `Class`. Before the `Class` can be used it must be resolved.

This method assigns a default `ProtectionDomain` to the newly defined class. The `getPermissions` method of the `ProtectionDomain` always returns `null`. The default protection domain is created on the first invocation of `defineClass`, and re-used on subsequent invocations.

To assign a specific `ProtectionDomain` to the class, use the `defineClass` method that takes a `ProtectionDomain` as one of its arguments.

This method defines a package in this class loader corresponding to the package of the `Class` (if such a package has not already been defined in this class loader). The name of the defined package is derived from the binary name of the class specified by the byte array `b`. Other properties of the defined package are as specified by `Package`.

Parameters:

`name` - The expected binary name of the class, or `null` if not known

`b` - The bytes that make up the class data. The bytes in positions `off` through `off+len-1` should have the format of a valid class file as defined by *The Java Virtual Machine Specification*.

`off` - The start offset in `b` of the class data

`len` - The length of the class data

Returns:

The `Class` object that was created from the specified class data.

Throws:

`ClassFormatError` - If the data did not contain a valid class

`IndexOutOfBoundsException` - If either `off` or `len` is negative, or if `off+len` is greater than `b.length`.

`SecurityException` - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class (which is unsigned), or if `name` begins with "`java.`".

Since:

1.1

See Also:

`loadClass(String, boolean)`,
`resolveClass(Class)`,
`CodeSource`,
`SecureClassLoader`

defineClass

```
protected final Class<?> defineClass
(String name,
 byte[] b,
 int off,
 int len,
 ProtectionDomain protectionDomain)
                                throws ClassFormatError
```

Converts an array of bytes into an instance of class `Class`, with a given `ProtectionDomain`.

If the given `ProtectionDomain` is `null`, then a default protection domain will be assigned to the class as specified in the documentation for `defineClass(String, byte[], int, int)`. Before the class can be used it must be resolved.

The first class defined in a package determines the exact set of certificates that all subsequent classes defined in that package must contain. The set of

certificates for a class is obtained from the CodeSource within the ProtectionDomain of the class. Any classes added to that package must contain the same set of certificates or a SecurityException will be thrown. Note that if name is null, this check is not performed. You should always pass in the binary name of the class you are defining as well as the bytes. This ensures that the class you are defining is indeed the class you think it is.

If the specified name begins with "java.", it can only be defined by the platform class loader or its ancestors; otherwise SecurityException will be thrown. If name is not null, it must be equal to the binary name of the class specified by the byte array b, otherwise a NoClassDefFoundError will be thrown.

This method defines a package in this class loader corresponding to the package of the Class (if such a package has not already been defined in this class loader). The name of the defined package is derived from the binary name of the class specified by the byte array b. Other properties of the defined package are as specified by Package.

Parameters:

name - The expected binary name of the class, or null if not known

b - The bytes that make up the class data. The bytes in positions off through off+len-1 should have the format of a valid class file as defined by *The Java Virtual Machine Specification*.

off - The start offset in b of the class data

len - The length of the class data

protectionDomain - The ProtectionDomain of the class

Returns:

The Class object created from the data, and ProtectionDomain.

Throws:

ClassFormatError - If the data did not contain a valid class

NoClassDefFoundError - If name is not null and not equal to the binary name of the class specified by b

IndexOutOfBoundsException - If either off or len is negative, or if off+len is greater than b.length.

SecurityException - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class, or if name begins with "java." and this class loader is not the platform class loader or its ancestor.

defineClass

```
protected final Class<?> defineClass
(String name,
 ByteBuffer b,
 ProtectionDomain protectionDomain)
                                throws ClassFormatError
```

Converts a `ByteBuffer` into an instance of class `Class`, with the given `ProtectionDomain`. If the given `ProtectionDomain` is `null`, then a default protection domain will be assigned to the class as specified in the documentation for `defineClass(String, byte[], int, int)`. Before the class can be used it must be resolved.

The rules about the first class defined in a package determining the set of certificates for the package, the restrictions on class names, and the defined package of the class are identical to those specified in the documentation for `defineClass(String, byte[], int, int, ProtectionDomain)`.

An invocation of this method of the form *cl*`.defineClass(`*name*`,` *bBuffer*`,` *pd*`)` yields exactly the same result as the statements

```
...
byte[] temp = new byte[bBuffer.remaining()];
bBuffer.get(temp);
return cl.defineClass(name, temp, 0, temp.length, pd);
```

Parameters:

`name` - The expected binary name. of the class, or `null` if not known

`b` - The bytes that make up the class data. The bytes from positions `b.position()` through `b.position() + b.limit() -1` should have the format of a valid class file as defined by *The Java Virtual Machine Specification*.

`protectionDomain` - The `ProtectionDomain` of the class, or `null`.

Returns:

The `Class` object created from the data, and `ProtectionDomain`.

Throws:

`ClassFormatError` - If the data did not contain a valid class.

`NoClassDefFoundError` - If `name` is not `null` and not equal to the binary name of the class specified by `b`

`SecurityException` - If an attempt is made to add this class to a package that contains classes that were signed by a different set of certificates than this class, or if `name` begins with `"java."`.

Since:

1.5

See Also:

`defineClass(String, byte[], int, int, ProtectionDomain)`

resolveClass

```
protected final void resolveClass(Class<?> c)
```

Links the specified class. This (misleadingly named) method may be used by a class loader to link a class. If the class `c` has already been linked, then this method simply returns. Otherwise, the class is linked as described in the "Execution" chapter of *The Java Language Specification*.

Parameters:

`c` - The class to link

Throws:

`NullPointerException` - If `c` is `null`.

See Also:

`defineClass(String, byte[], int, int)`

findSystemClass

```
protected final Class<?> findSystemClass(String name)
                                  throws ClassNotFoundException
```

Finds a class with the specified binary name, loading it if necessary.

This method loads the class through the system class loader (see `getSystemClassLoader()`). The `Class` object returned might have more than one `ClassLoader` associated with it. Subclasses of `ClassLoader` need not usually invoke this method, because most class loaders need to override just `findClass(String)`.

Parameters:

`name` - The binary name of the class

Returns:

The `Class` object for the specified `name`

Throws:

`ClassNotFoundException` - If the class could not be found

See Also:

`ClassLoader(ClassLoader)`, `getParent()`

findLoadedClass

```
protected final Class<?> findLoadedClass(String name)
```

Returns the class with the given binary name if this loader has been recorded by the Java virtual machine as an initiating loader of a class with that binary name. Otherwise `null` is returned.

Parameters:

`name` - The binary name of the class

Returns:

The `Class` object, or `null` if the class has not been loaded

Since:

1.1

setSigners

```
protected final void setSigners(Class<?> c,
                                Object[] signers)
```

Sets the signers of a class. This should be invoked after defining a class.

Parameters:

`c` - The `Class` object

`signers` - The signers for the class

Since:

1.1

findResource

```
protected URL findResource(String moduleName,
                           String name)
                    throws IOException
```

Returns a URL to a resource in a module defined to this class loader. Class loader implementations that support loading from modules should override this method.

API Note:

This method is the basis for the `Class.getResource`, `Class.getResourceAsStream`, and `Module.getResourceAsStream` methods. It is not subject to the rules for encapsulation specified by `Module.getResourceAsStream`.

Implementation Requirements:

The default implementation attempts to find the resource by invoking `findResource(String)` when the `moduleName` is `null`. It otherwise returns `null`.

Parameters:

`moduleName` - The module name; or `null` to find a resource in the unnamed module for this class loader

`name` - The resource name

Returns:

A URL to the resource; `null` if the resource could not be found, a URL could not be constructed to locate the resource, or there isn't a module of the given name defined to the class loader.

Throws:

`IOException` - If I/O errors occur

Since:

9

See Also:

`ModuleReader.find(String)`

getResource

```
public URL getResource(String name)
```

Finds the resource with the given name. A resource is some data (images, audio, text, etc) that can be accessed by class code in a way that is independent of the location of the code.

The name of a resource is a '`/`'-separated path name that identifies the resource.

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with "`.class`", this method will only find resources in packages of named modules when the package is `opened` unconditionally (even if the caller of this method is in the same module as the resource).

API Note:

Where several modules are defined to the same class loader, and where more than one module contains a resource with the given name, then the ordering that modules are searched is not specified and may be very unpredictable. When overriding this method it is recommended that an implementation ensures that any delegation is consistent with the `getResources(String)` method.

Implementation Requirements:

The default implementation will first search the parent class loader for the resource; if the parent is `null` the path of the class loader built into the virtual machine is searched. If not found, this method will invoke `findResource(String)` to find the resource.

Parameters:

`name` - The resource name

Returns:

`URL` object for reading the resource; `null` if the resource could not be found, a `URL` could not be constructed to locate the resource, or the resource is in a package that is not opened unconditionally.

Throws:

`NullPointerException` - If `name` is `null`

Since:

1.1

getResources

```
public Enumeration<URL> getResources(String name)
                                throws IOException
```

Finds all the resources with the given name. A resource is some data (images, audio, text, etc) that can be accessed by class code in a way that is independent of the location of the code.

The name of a resource is a `/`-separated path name that identifies the resource.

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with `".class"`, this method will only find resources in packages of named modules when the package is `opened` unconditionally (even if the caller of this method is in the same module as the resource).

API Note:

Where several modules are defined to the same class loader, and where more than one module contains a resource with the given name, then the ordering is not specified and may be very unpredictable. When overriding this method it is recommended that an implementation ensures that any delegation is consistent with the `getResource(String)` method. This should ensure that the first element returned by the Enumeration's `nextElement` method is the same resource that the `getResource(String)` method would return.

Implementation Requirements:

The default implementation will first search the parent class loader for the resource; if the parent is `null` the path of the class loader built into the virtual machine is searched. It then invokes `findResources(String)` to find the resources with the name in this class loader. It returns an enumeration whose elements are the URLs found by searching the parent class loader followed by the elements found with `findResources`.

Parameters:

`name` - The resource name

Returns:

An enumeration of `URL` objects for the resource. If no resources could be found, the enumeration will be empty. Resources for which a `URL` cannot be constructed, or are in a package that is not opened unconditionally, are not returned in the enumeration.

Throws:

`IOException` - If I/O errors occur

`NullPointerException` - If `name` is `null`

Since:

1.2

resources

```
public Stream<URL> resources(String name)
```

Returns a stream whose elements are the URLs of all the resources with the given name. A resource is some data (images, audio, text, etc) that can be accessed by class code in a way that is independent of the location of the code.

The name of a resource is a `/`-separated path name that identifies the resource.

The resources will be located when the returned stream is evaluated. If the evaluation results in an `IOException` then the I/O exception is wrapped in an `UncheckedIOException` that is then thrown.

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with `".class"`, this method will only find resources in packages of named modules when the package is `opened` unconditionally (even if the caller of this method is in the same module as the resource).

API Note:

When overriding this method it is recommended that an implementation ensures that any delegation is consistent with the `getResource(String)` method. This should ensure that the first element returned by the stream is the same resource that the `getResource(String)` method would return.

Implementation Requirements:

The default implementation invokes `getResources` to find all the resources with the given name and returns a stream with the elements in the enumeration as the source.

Parameters:

`name` - The resource name

Returns:

A stream of resource `URL` objects. If no resources could be found, the stream will be empty. Resources for which a `URL` cannot be constructed, or are in a package that is not opened unconditionally, will not be in the stream.

Throws:

`NullPointerException` - If `name` is `null`

Since:

9

findResource

```
protected URL findResource(String name)
```

Finds the resource with the given name. Class loader implementations should override this method.

For resources in named modules then the method must implement the rules for encapsulation specified in the `Module` `getResourceAsStream` method. Additionally, it must not find non-`".class"` resources in packages of named modules unless the package is `opened` unconditionally.

Implementation Requirements:

The default implementation returns `null`.

Parameters:

`name` - The resource name

Returns:

`URL` object for reading the resource; `null` if the resource could not be found, a `URL` could not be constructed to locate the resource, or the resource is in a package that is not opened unconditionally.

Since:

1.2

findResources

```
protected Enumeration<URL> findResources(String name)
                                   throws IOException
```

Returns an enumeration of URL objects representing all the resources with the given name. Class loader implementations should override this method.

For resources in named modules then the method must implement the rules for encapsulation specified in the Module getResourceAsStream method. Additionally, it must not find non-".class" resources in packages of named modules unless the package is opened unconditionally.

Implementation Requirements:

The default implementation returns an enumeration that contains no elements.

Parameters:

name - The resource name

Returns:

An enumeration of URL objects for the resource. If no resources could be found, the enumeration will be empty. Resources for which a URL cannot be constructed, or are in a package that is not opened unconditionally, are not returned in the enumeration.

Throws:

IOException - If I/O errors occur

Since:

1.2

registerAsParallelCapable

```
protected static boolean registerAsParallelCapable()
```

Registers the caller as parallel capable. The registration succeeds if and only if all of the following conditions are met:

1. no instance of the caller has been created
2. all of the super classes (except class Object) of the caller are registered as parallel capable

Note that once a class loader is registered as parallel capable, there is no way to change it back.

In cases where this method is called from a context where the caller is not a subclass of ClassLoader or there is no caller frame on the stack (e.g. when called directly from a JNI attached thread), IllegalCallerException is thrown.

Returns:

`true` if the caller is successfully registered as parallel capable and `false` if otherwise.

Throws:

`IllegalCallerException` - if the caller is not a subclass of `ClassLoader`

Since:

1.7

See Also:

`isRegisteredAsParallelCapable()`

isRegisteredAsParallelCapable

```
public final boolean isRegisteredAsParallelCapable()
```

Returns `true` if this class loader is registered as parallel capable, otherwise `false`.

Returns:

`true` if this class loader is parallel capable, otherwise `false`.

Since:

9

See Also:

`registerAsParallelCapable()`

getSystemResource

```
public static URL getSystemResource(String name)
```

Find a resource of the specified name from the search path used to load classes. This method locates the resource through the system class loader (see `getSystemClassLoader()`).

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with `".class"`, this method will only find resources in packages of named modules when the package is `opened` unconditionally.

Parameters:

`name` - The resource name

Returns:

A `URL` to the resource; `null` if the resource could not be found, a URL could not be constructed to locate the resource, or the resource is in a package that is not opened unconditionally.

Since:

1.1

getSystemResources

```
public static Enumeration<URL> getSystemResources(String name)
                                                  throws IOException
```

Finds all resources of the specified name from the search path used to load classes. The resources thus found are returned as an `Enumeration` of `URL` objects.

The search order is described in the documentation for `getSystemResource(String)`.

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with `".class"`, this method will only find resources in packages of named modules when the package is `opened` unconditionally.

Parameters:

`name` - The resource name

Returns:

An enumeration of `URL` objects for the resource. If no resources could be found, the enumeration will be empty. Resources for which a `URL` cannot be constructed, or are in a package that is not opened unconditionally, are not returned in the enumeration.

Throws:

`IOException` - If I/O errors occur

Since:

1.2

getResourceAsStream

```
public InputStream getResourceAsStream(String name)
```

Returns an input stream for reading the specified resource.

The search order is described in the documentation for `getResource(String)`.

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with `".class"`, this method will only find resources in packages of named modules when the package is `opened` unconditionally.

Parameters:

`name` - The resource name

Returns:

An input stream for reading the resource; `null` if the resource could not be found, or the resource is in a package that is not opened unconditionally.

Throws:

`NullPointerException` - If `name` is `null`

Since:

1.1

getSystemResourceAsStream

```
public static InputStream getSystemResourceAsStream(String name)
```

Open for reading, a resource of the specified name from the search path used to load classes. This method locates the resource through the system class loader (see `getSystemClassLoader()`).

Resources in named modules are subject to the encapsulation rules specified by `Module.getResourceAsStream`. Additionally, and except for the special case where the resource has a name ending with `".class"`, this method will only find resources in packages of named modules when the package is `opened` unconditionally.

Parameters:

`name` - The resource name

Returns:

An input stream for reading the resource; `null` if the resource could not be found, or the resource is in a package that is not opened unconditionally.

Since:

1.1

getParent

```
public final ClassLoader getParent()
```

Returns the parent class loader for delegation. Some implementations may use `null` to represent the bootstrap class loader. This method will return `null` in such implementations if this class loader's parent is the bootstrap class loader.

Returns:

The parent `ClassLoader`

Since:

1.2

getUnnamedModule

```
public final Module getUnnamedModule()
```

Returns the unnamed `Module` for this class loader.

Returns:

The unnamed Module for this class loader

Since:

9

See Also:

`Module.isNamed()`

getPlatformClassLoader

```
public static ClassLoader getPlatformClassLoader()
```

Returns the platform class loader. All platform classes are visible to the platform class loader.

Implementation Note:

The name of the builtin platform class loader is `"platform"`.

Returns:

The platform `ClassLoader`.

Since:

9

getSystemClassLoader

```
public static ClassLoader getSystemClassLoader()
```

Returns the system class loader. This is the default delegation parent for new `ClassLoader` instances, and is typically the class loader used to start the application.

This method is first invoked early in the runtime's startup sequence, at which point it creates the system class loader. This class loader will be the context class loader for the main application thread (for example, the thread that invokes the `main` method of the main class).

The default system class loader is an implementation-dependent instance of this class.

If the system property "`java.system.class.loader`" is defined when this method is first invoked then the value of that property is taken to be the name of a class that will be returned as the system class loader. The class is loaded using the default system class loader and must define a public constructor that takes a single parameter of type `ClassLoader` which is used as the delegation parent. An instance is then created using this constructor with the default system class loader as the parameter. The resulting class loader is defined to be the system class loader. During construction, the class loader should take great care to avoid calling `getSystemClassLoader()`. If circular initialization of the system class loader is detected then an `IllegalStateException` is thrown.

Implementation Note:

The system property to override the system class loader is not examined until the VM is almost fully initialized. Code that executes this method during startup should take care not to cache the return value until the system is fully initialized.

The name of the built-in system class loader is "`app`". The system property "`java.class.path`" is read during early initialization of the VM to determine the class path. An empty value of "`java.class.path`" property is interpreted differently depending on whether the initial module (the module containing the main class) is named or unnamed: If named, the built-in system class loader will have no class path and will search for classes and resources using the application module path; otherwise, if unnamed, it will set the class path to the current working directory.

JAR files on the class path may contain a `Class-Path` manifest attribute to specify dependent JAR files to be included in the class path. `Class-Path` entries must meet certain conditions for validity (see the JAR File Specification for details). Invalid `Class-Path` entries are ignored. For debugging purposes, ignored entries can be printed to the console if the `jdk.net.URLClassPath.showIgnoredClassPathEntries` system property is set to `true`.

Returns:

The system `ClassLoader`

Throws:

`IllegalStateException` - If invoked recursively during the construction of the class loader specified by the "`java.system.class.loader`" property.

`Error` - If the system property "`java.system.class.loader`" is defined but the named class could not be loaded, the provider class does not define the required constructor, or an exception is thrown by that constructor when it is invoked. The underlying cause of the error can be retrieved via the `Throwable.getCause()` method.

definePackage

```
protected Package definePackage(String name,
                                String specTitle,
                                String specVersion,
                                String specVendor,
                                String implTitle,
                                String implVersion,
                                String implVendor,
                                URL sealBase)
```

Defines a package by name in this `ClassLoader`.

Package names must be unique within a class loader and cannot be redefined or changed once created.

If a class loader wishes to define a package with specific properties, such as version information, then the class loader should call this `definePackage` method before calling `defineClass`. Otherwise, the `defineClass` method will define a package in this class loader corresponding to the package of the newly defined class; the properties of this defined package are specified by `Package`.

API Note:

A class loader that wishes to define a package for classes in a JAR typically uses the specification and implementation titles, versions, and vendors from the JAR's manifest. If the package is specified as sealed in the JAR's manifest, the `URL` of the JAR file is typically used as the `sealBase`. If classes of package `'p'` defined by this class loader are loaded from multiple JARs, the `Package` object may contain different information depending on the first class of package `'p'` defined and which JAR's manifest is read first to explicitly define package `'p'`.

It is strongly recommended that a class loader does not call this method to explicitly define packages in *named modules*; instead, the package will be automatically defined when a class is being defined. If it is desirable to define `Package` explicitly, it should ensure that all packages in a named module are defined with the properties specified by `Package`. Otherwise, some `Package` objects in a named module may be for example sealed with different seal base.

Parameters:

`name` - The package name

`specTitle` - The specification title

`specVersion` - The specification version

`specVendor` - The specification vendor

`implTitle` - The implementation title

`implVersion` - The implementation version

`implVendor` - The implementation vendor

`sealBase` - If not `null`, then this package is sealed with respect to the given code source `URL` object. Otherwise, the package is not sealed.

Returns:

The newly defined `Package` object

Throws:

`NullPointerException` - if `name` is `null`.

`IllegalArgumentException` - if a package of the given `name` is already defined by this class loader

See *Java Virtual Machine Specification*:

5.3 Creation and Loading

Since:

1.2

See Also:

The JAR File Specification: Package Sealing

getDefinedPackage

```
public final Package getDefinedPackage(String name)
```

Returns a `Package` of the given `name` that has been defined by this class loader.

Parameters:

`name` - The package name

Returns:

The `Package` of the given name that has been defined by this class loader, or `null` if not found

Throws:

`NullPointerException` - if `name` is `null`.

See *Java Virtual Machine Specification*:

5.3 Creation and Loading

Since:

9

getDefinedPackages

```
public final Package[] getDefinedPackages()
```

Returns all of the Packages that have been defined by this class loader. The returned array has no duplicated Packages of the same name.

API Note:

This method returns an array rather than a Set or Stream for consistency with the existing getPackages() method.

Returns:

The array of Package objects that have been defined by this class loader; or a zero length array if no package has been defined by this class loader.

See *Java Virtual Machine Specification*:

5.3 Creation and Loading

Since:

9

getPackage

```
@Deprecated(since="9")
protected Package getPackage(String name)
```

Deprecated.
If multiple class loaders delegate to each other and define classes with the same package name, and one such loader relies on the lookup behavior of getPackage to return a Package from a parent loader, then the properties exposed by the Package may not be as expected in the rest of the program. For example, the Package will only expose annotations from the package-info.class file defined by the parent loader, even if annotations exist in a package-info.class file defined by a child loader. A more robust approach is to use the getDefinedPackage(String) method which returns a Package for the specified class loader.

Finds a package by name in this class loader and its ancestors.

If this class loader defines a Package of the given name, the Package is returned. Otherwise, the ancestors of this class loader are searched recursively (parent by parent) for a Package of the given name.

API Note:

The platform class loader may delegate to the application class loader but the application class loader is not its ancestor. When invoked on the platform class loader, this method will not find packages defined to the application class loader.

Parameters:

`name` - The package name

Returns:

The `Package` of the given name that has been defined by this class loader or its ancestors, or `null` if not found.

Throws:

`NullPointerException` - if `name` is `null`.

Since:

1.2

See Also:

`getDefinedPackage(String)`

getPackages

```
protected Package[] getPackages()
```

Returns all of the `Packages` that have been defined by this class loader and its ancestors. The returned array may contain more than one `Package` object of the same package name, each defined by a different class loader in the class loader hierarchy.

API Note:

The `platform class loader` may delegate to the application class loader. In other words, packages in modules defined to the application class loader may be visible to the platform class loader. On the other hand, the application class loader is not its ancestor and hence when invoked on the platform class loader, this method will not return any packages defined to the application class loader.

Returns:

The array of `Package` objects that have been defined by this class loader and its ancestors

Since:

1.2

See Also:

`getDefinedPackages()`

findLibrary

```
protected String findLibrary(String libname)
```

Returns the absolute path name of a native library. The VM invokes this method to locate the native libraries that belong to classes loaded with this

class loader. If this method returns `null`, the VM searches the library along the path specified as the "`java.library.path`" property.

Parameters:

`libname` - The library name

Returns:

The absolute path of the native library

Since:

1.2

See Also:

`System.loadLibrary(String)` RESTRICTED,
`System.mapLibraryName(String)`

setDefaultAssertionStatus

```
public void setDefaultAssertionStatus(boolean enabled)
```

Sets the default assertion status for this class loader. This setting determines whether classes loaded by this class loader and initialized in the future will have assertions enabled or disabled by default. This setting may be overridden on a per-package or per-class basis by invoking `setPackageAssertionStatus(String, boolean)` or `setClassAssertionStatus(String, boolean)`.

Parameters:

`enabled` - `true` if classes loaded by this class loader will henceforth have assertions enabled by default, `false` if they will have assertions disabled by default.

Since:

1.4

setPackageAssertionStatus

```
public void setPackageAssertionStatus(String packageName,
                                       boolean enabled)
```

Sets the package default assertion status for the named package. The package default assertion status determines the assertion status for classes initialized in the future that belong to the named package or any of its "subpackages".

A subpackage of a package named p is any package whose name begins with "`p.`". For example, `javax.swing.text` is a subpackage of `javax.swing`, and both `java.util` and `java.lang.reflect` are subpackages of `java`.

In the event that multiple package defaults apply to a given class, the package default pertaining to the most specific package takes precedence over the others. For example, if `javax.lang` and `javax.lang.reflect` both have package defaults associated with them, the latter package default applies to classes in `javax.lang.reflect`.

Package defaults take precedence over the class loader's default assertion status, and may be overridden on a per-class basis by invoking `setClassAssertionStatus(String, boolean)`.

Parameters:

`packageName` - The name of the package whose package default assertion status is to be set. A `null` value indicates the unnamed package that is "current" (see section §7.4.2 of *The Java Language Specification*.)

`enabled` - `true` if classes loaded by this classloader and belonging to the named package or any of its subpackages will have assertions enabled by default, `false` if they will have assertions disabled by default.

Since:

1.4

setClassAssertionStatus

```
public void setClassAssertionStatus(String className,
                                     boolean enabled)
```

Sets the desired assertion status for the named top-level class in this class loader and any nested classes contained therein. This setting takes precedence over the class loader's default assertion status, and over any applicable per-package default. This method has no effect if the named class has already been initialized. (Once a class is initialized, its assertion status cannot change.)

If the named class is not a top-level class, this invocation will have no effect on the actual assertion status of any class.

Parameters:

`className` - The fully qualified class name of the top-level class whose assertion status is to be set.

`enabled` - `true` if the named class is to have assertions enabled when (and if) it is initialized, `false` if the class is to have assertions disabled.

Since:

1.4

clearAssertionStatus

```
public void clearAssertionStatus()
```

Sets the default assertion status for this class loader to `false` and discards any package defaults or class assertion status settings associated with the class loader. This method is provided so that class loaders can be made to ignore any command line or persistent assertion status settings and "start with a clean slate."

Since:

1.4

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.